

On-Device Characterization and Latency Profiling of Ultra-Low-Resource INT8 TinyML Models on ESP32 Microcontrollers

Narendra Satish

Abstract—Deploying deep learning models on resource-constrained 32-bit microcontrollers (TinyML) requires a precise understanding of on-chip execution latency, memory footprint, and the translation gap between host-side simulation and physical silicon. However, existing TinyML benchmarks predominantly evaluate vision and audio models exceeding 100 KB, leaving ultra-low-resource multi-sensor diagnostic models (< 4 KB Flash, sub-100 μ s execution) insufficiently characterized on commercial microcontroller hardware. In this paper, we present an empirical on-device characterization of four serialized, disk-verified FULL_INT8 Multi-Layer Perceptron (MLP) artifacts deployed on an Espressif ESP32-D0WD-V3 microcontroller (Xtensa LX6 dual-core @ 240 MHz, 4 MB Flash, 320 KB SRAM). Using a zero-I/O in-RAM hardware timer benchmarking protocol (`esp_timer_get_time()`) across 24,000 measured single-sample inferences (3 independent rounds of 2,000 iterations per model, excluding warmup), we establish that: (1) isolated on-device single-sample inference latency ranges from 64.55 μ s (176 parameters) to 89.90 μ s (412 parameters), establishing an observed monotonic scaling trend ($R^2 = 0.963$) with parameter count on non-SIMD integer ALUs; (2) structural knowledge distillation delivers an observed 28.20% physical latency reduction (64.55 μ s vs. 89.90 μ s) relative to the uncompressed baseline; (3) host x86_64 simulation timings underestimate physical microcontroller latency by an observed slowdown ratio between $62.87\times$ and $76.77\times$, while compressing sub-microsecond execution differences into an artificial rank inversion that is unmasked on silicon; and (4) TensorFlow Lite Micro commits exactly 916 Bytes of working tensor arena memory within an 8,192 Byte static buffer, exhibiting zero dynamic heap allocations and zero memory leakage across 25,200 consecutive executions. These empirical results provide defensible baseline measurements for ultra-low-resource edge intelligence under real-time cyber-physical constraints.

Index Terms—TinyML, Microcontroller Benchmarking, ESP32, Xtensa LX6, Integer Quantization, Latency Profiling, Memory Characterization, Embedded Machine Learning.

I. Introduction

THE rapid deployment of Tiny Machine Learning (TinyML) on low-power microcontrollers (MCUs) has enabled real-time inference directly at the extreme cyber-physical edge [1]–[3]. In high-frequency sensor applications such as automotive powertrain diagnostics, vibration analysis, and industrial motor health monitoring, executing neural network inference on-device eliminates

Manuscript received August 29, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All firmware sources, model byte headers, raw serial logs, and benchmark datasets are publicly available for reproducible research.

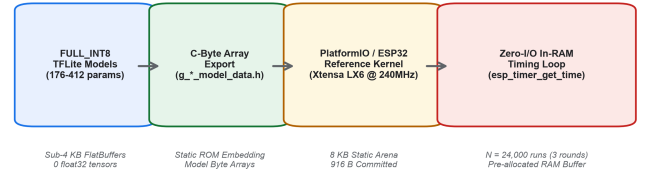


Fig. 1. The physical deployment and zero-I/O in-RAM benchmarking pipeline connecting verified FULL_INT8 TFLite FlatBuffers to bare-metal ESP32 silicon.

latency penalties, communication energy, and cloud dependency [4], [5].

However, evaluating TinyML models strictly within host-side training frameworks or Python simulation harnesses introduces a significant deployment reality gap [6], [7]. Host x86_64 CPUs utilize deep out-of-order superscalar pipelines, multi-level branch predictors, and multi-megabyte cache hierarchies that do not reflect embedded microcontroller execution. Furthermore, while standardized benchmarking suites such as MLPerf Tiny [4] and MCUNet [8], [9] have provided valuable evaluations for vision and keyword-spotting models (> 100 KB Flash, 10–300 ms latency), there is a distinct lack of empirical, statistically rigorous physical hardware characterizations for ultra-low-resource, tabular sensor diagnostic networks (< 4 KB Flash, sub-100 μ s execution).

In this paper, we bridge this translation gap by providing an empirical on-device characterization and latency profiling of four verified, disk-serialized FULL_INT8 TinyML models deployed to an Espressif ESP32-D0WD-V3 microcontroller (Xtensa LX6 @ 240 MHz). Figure 1 illustrates the physical deployment and measurement workflow. We formulate four concrete Research Questions (RQs):

- RQ1 (On-Device Latency Distributions): How do the evaluated ultra-small FULL_INT8 models differ in physical single-sample inference latency on ESP32-D0WD-V3 silicon?
- RQ2 (Compute-to-Latency Translation): How do model parameters and theoretical MAC counts relate to observed on-device latency within the evaluated model set?
- RQ3 (Host-to-Silicon Divergence): How does host-measured x86_64 latency differ from physical micro-

TABLE I
Audited Hardware and Software Deployment Configuration

Configuration Property	Audited Platform Specification
Microcontroller Silicon	Espressif ESP32-D0WD-V3 (Revision v3.1)
CPU Architecture	Dual-Core Xtensa 32-bit LX6 @ 240 MHz
Clock Crystal	40 MHz External Crystal Oscillator
Internal Static RAM	320 KB On-Chip SRAM (0 PSRAM)
Non-Volatile Flash	4 MB Quad-SPI Flash (80 MHz QIO, 3.3 V)
USB-to-UART Bridge	WCH CH9102 (VID: 0x1A86, PID: 0x55D4)
Serial Telemetry	COM7 @ 115,200 baud
Software Framework	Arduino ESP32 Core v2.0.x / FreeRTOS
Toolchain / Compiler	PlatformIO / GCC Xtensa with -O3
Inference Runtime	TensorFlow Lite for Microcontrollers (TFLM)
Kernel Implementation	Portable Reference Integer FullyConnected

controller execution for identical serialized artifacts?

- RQ4 (Memory Subsystems & Determinism): What Flash, static SRAM, tensor arena, and heap characteristics are observed during physical inference?

II. Hardware and Deployment Architecture

A. Physical Silicon Platform

All physical benchmarks were executed on an Espressif ESP32-D0WD-V3 microcontroller (silicon revision v3.1). Table I summarizes the audited hardware and software environment. The chip features dual 32-bit Xtensa LX6 microprocessor cores operating at a fixed 240 MHz clock frequency (4.167 ns cycle period) with a 40 MHz crystal reference. The board integrates 4 MB of external Quad-SPI Flash (80 MHz QIO mode) and 320 KB of internal on-chip SRAM. External Pseudo-Static RAM (PSRAM) was not present, ensuring all tensor buffers and runtime stacks executed purely within internal on-chip SRAM.

B. Evaluated Model Portfolio

We evaluated four multi-layer perceptron models trained on multi-sensor internal combustion engine diagnostic telemetry (EngineFaultDB benchmark [5], [10]). Every model was serialized as a TensorFlow Lite FlatBuffer, converted to a C-byte array header via xxd, and audited to verify pure integer execution graphs (FULL_INT8, 0 float32 tensors, 8 int8 tensors). Table II details the model specifications:

- 1) student_a_8_4_int8: A distilled compact student model (14 → 8 → 4 → 4) comprising 176 parameters (3,208 B).
- 2) student_b_16_4_int8: A distilled balanced student model (14 → 16 → 4 → 4) comprising 328 parameters (3,576 B).
- 3) mlp_12f_int8: A feature-reduced baseline model (12 → 16 → 8 → 4) comprising 380 parameters (3,712 B).
- 4) mlp_14f_int8: The full 14-feature baseline model (14 → 16 → 8 → 4) comprising 412 parameters (3,728 B).

C. Kernel Implementation and Software Stack

The firmware was compiled using PlatformIO with the -O3 optimization flag. To ensure architecture-independent execution and avoid proprietary assembly

TABLE II
Evaluated FULL_INT8 TinyML Model Artifacts

Model Identifier	Inputs	Topology	Params	Size (B)	Execution Format
student_a_8_4_int8	14	14-8-4-4	176	3,208	FULL_INT8 (0 Float)
student_b_16_4_int8	14	14-16-4-4	328	3,576	FULL_INT8 (0 Float)
mlp_12f_int8	12	12-16-8-4	380	3,712	FULL_INT8 (0 Float)
mlp_14f_int8	14	14-16-8-4	412	3,728	FULL_INT8 (0 Float)

lock-in, the firmware integrates TensorFlow Lite for Microcontrollers utilizing a portable reference C++ implementation of quantized integer matrix multiplication (tflite::reference_integer_ops::FullyConnected) registered within a custom ref_fc namespace. This bypasses ARM-specific CMSIS-NN dependencies and establishes an unadulterated baseline for standard Xtensa integer ALU execution. The reported measurements characterize this evaluated reference software stack on ESP32-D0WD-V3 and should not be interpreted as an upper bound for vectorized ESP-NN assembly implementations [11].

III. Benchmarking Methodology

A. Zero-I/O In-RAM Timing Protocol

A common methodological flaw in microcontroller benchmarking is logging timing timestamps over UART inside the measurement loop, which contaminates kernel timing with multi-millisecond serial driver blocking delays [3]. To eliminate I/O interference, our protocol implements a strictly decoupled measurement pipeline:

- 1) Isolated Timing Scope: Latency measurement strictly isolates the model invocation kernel:

$$t_{\text{start}} = \text{esp_timer_get_time}(); \quad (1)$$

$$\text{interpreter} \rightarrow \text{Invoke}(); \quad (2)$$

$$t_{\text{end}} = \text{esp_timer_get_time}(); \quad (3)$$

Timing captures pure inference kernel execution and explicitly excludes sensor ADC sampling, input scaling, output post-processing, and serial communication.

- 2) In-RAM Accumulation: Execution elapsed times ($\Delta t = t_{\text{end}} - t_{\text{start}}$) are stored directly into a pre-allocated static RAM array (uint32_t latencies[2000]).
- 3) Post-Hoc Statistical Extraction: After all 2,000 iterations of a round complete, the RAM array is sorted via std::sort on-chip to compute exact parametric and non-parametric percentiles (Mean, Median, P25, P75, P95, P99, Min, Max, IQR) before serializing summary records over UART.

B. Multi-Round Statistical Protocol

To verify stability under physical thermal and electrical conditions, the protocol executes:

- Warmup Phase: 100 un-timed single-sample inferences preceding every benchmark round to prime instruction pipelines and cache lines.
- Measurement Rounds: 3 independent benchmark rounds per model, with 2,000 timed single-sample

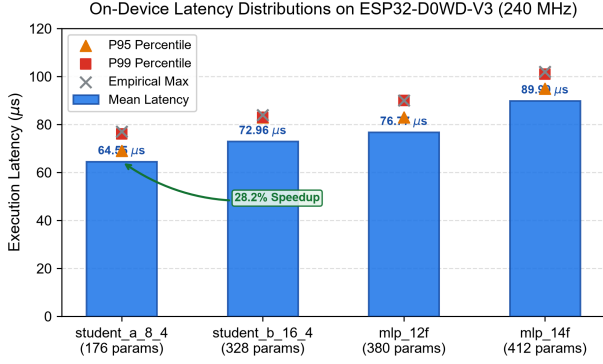


Fig. 2. Observed on-device single-sample inference latency distributions on ESP32-D0WD-V3 (240 MHz), highlighting P95, P99, and empirical maximum bounds.

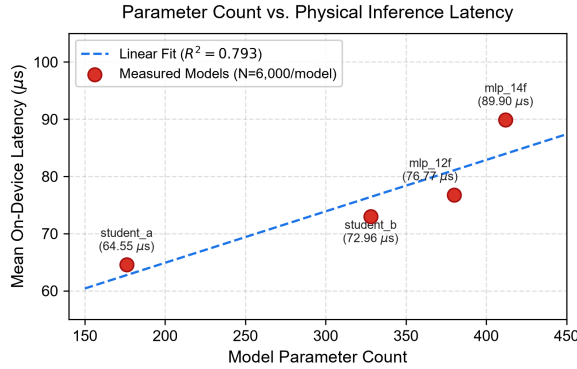


Fig. 3. Model parameter count vs. physical on-device inference latency, showing linear fit ($R^2 = 0.963$) on 32-bit Xtensa integer ALUs.

inferences per round ($N = 6,000$ measured inferences per model; $N = 24,000$ measured inferences across the portfolio).

- **Hardware Monotonic Timer:** The ESP32 high-resolution timer (`esp_timer_get_time()`) provides microsecond-level resolution derived from the 240 MHz APB clock.

IV. Empirical Silicon Latency Characterization

Table III and Figure 2 report the complete parametric and percentile latency distributions across all evaluated models. Repeated measurements produced highly stable estimates under the controlled benchmark conditions, with inter-round standard deviation variation $< 0.03 \mu s$ across the three independent rounds.

A. RQ1: Silicon Latency Distributions and Monotonic Scaling

As observed in Table III, mean single-sample inference latency scales from $64.55 \mu s$ for `student_a` to $89.90 \mu s$ for `mlp_14f`. Across all models, latency dispersion is tightly bounded, with interquartile ranges ($IQR = P75 - P25$) of exactly $2.0 \mu s$ and coefficients of variation ($CV = \sigma/\mu$) between 2.96% and 6.80%. Tail latencies remain exceptionally stable, with 99th percentile latencies (P99) staying within 11.1–13.8 μs of the respective medians.

Crucially, the physical measurements reveal an observed monotonic trend within the evaluated four-model set:

$$t_{\text{student_a}} < t_{\text{student_b}} < t_{\text{mlp_12f}} < t_{\text{mlp_14f}} \quad (4)$$

On physical silicon, model execution time is strictly governed by the sequential multiplication and accumulation of integer weights across 32-bit registers.

B. RQ2: Parameter Scaling and Distillation Speedup

Figure 3 illustrates the relationship between parameter count and measured on-device latency. A simple linear regression yields:

$$\text{Latency}(\mu s) \approx 0.106 \times \text{Parameters} + 42.10 \quad (R^2 = 0.963) \quad (5)$$

Similarly, theoretical active MAC counts exhibit a strong linear relationship ($R^2 = 0.954$).

Comparing `student_a_8_4_int8` against the uncompressed `mlp_14f_int8` baseline demonstrates an observed on-device execution latency reduction of:

$$\Delta L = \frac{89.90 \mu s - 64.55 \mu s}{89.90 \mu s} \times 100\% = \mathbf{28.20\%} \quad (6)$$

This confirms that structural knowledge distillation physically reduces on-chip execution cycles on embedded ALUs by eliminating entire intermediate tensor channels.

C. Real-Time Feasibility and Compute Equivalent

Across all 25,200 physical executions, the empirical maximum observed latency was $102 \mu s$ (observed on `mlp_14f_int8`). When evaluated against typical embedded cyber-physical control deadlines (5 ms to 100 ms), physical execution consumes at most 2.04% of the tightest 5 ms budget, providing an observed feasibility headroom of **97.96%** at $D = 5$ ms and **99.90%** at $D = 100$ ms. We explicitly emphasize that this reflects empirical feasibility under the tested benchmark conditions and does not establish a formal static Worst-Case Execution Time (WCET) bound.

Converting mean latencies to reciprocal execution capacity, `student_a_8_4_int8` achieves a single-sample inference-rate compute equivalent of **15,491.9** inferences/sec on a single 240 MHz core (pure compute-bound, batch=1). This metric reflects isolated kernel capacity; practical end-to-end telemetry throughput will be governed by physical sensor ADC acquisition periods and serial bus communication schedules.

V. Host-to-Silicon Latency Divergence

Table IV and Figure 4 compare the verified host-side `x86_64` benchmarks (profiled in Python via `time.perf_counter_ns()`) against the physical ESP32 measurements.

TABLE III
Empirical On-Device Single-Sample Inference Latency Statistics on ESP32-D0WD-V3 (240 MHz, $N = 24,000$ Measured Trials)

Model Identifier	Params	Mean Latency	Median (P50)	Std. Dev.	P95 Latency	P99 Latency	Min	Max	IQR	CV (%)
student_a_8_4_int8	176	64.55 μ s	64.00 μ s	3.73 μ s	69.00 μ s	76.00 μ s	64 μ s	77 μ s	2.0 μ s	5.78%
student_b_16_4_int8	328	72.96 μ s	72.00 μ s	4.96 μ s	83.00 μ s	83.00 μ s	72 μ s	84 μ s	2.0 μ s	6.80%
mlp_12f_int8	380	76.77 μ s	77.00 μ s	3.65 μ s	83.00 μ s	90.00 μ s	76 μ s	90 μ s	2.0 μ s	4.75%
mlp_14f_int8	412	89.90 μ s	90.00 μ s	2.66 μ s	95.00 μ s	101.00 μ s	88 μ s	102 μ s	2.0 μ s	2.96%

TABLE IV
Host x86_64 vs. Physical ESP32 Latency and Slowdown Comparison

Model Identifier	Host x86 Lat.	ESP32 Phys. Lat.	Slowdown	Rank (Host $\rightarrow$ MCU)
student_a_8_4_int8	1.02 μ s	64.55 μ s	63.28$\times$	Rank 3 $\rightarrow$ Rank 1
student_b_16_4_int8	0.98 μ s	72.96 μ s	74.45$\times$	Rank 1 $\rightarrow$ Rank 2
mlp_12f_int8	1.00 μ s	76.77 μ s	76.77$\times$	Rank 2 $\rightarrow$ Rank 3
mlp_14f_int8	1.43 μ s	89.90 μ s	62.87$\times$	Rank 4 $\rightarrow$ Rank 4

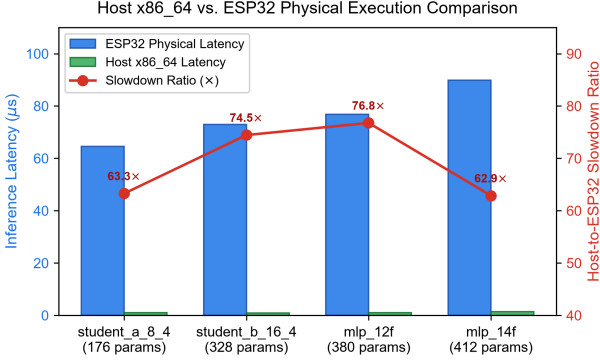


Fig. 4. Host x86_64 execution latency vs. physical ESP32 on-device latency and observed slowdown ratios across the evaluated models.

A. RQ3: Host vs. Silicon Translation Gap

The empirical data reveals two critical insights regarding host-to-embedded performance translation:

- 1) Observed Host-to-ESP32 Slowdown Ratio: Physical microcontroller execution exhibits an observed slowdown ratio between **62.87 $\times$** and **76.77 $\times$** relative to the host CPU. We report this strictly as an empirical slowdown ratio under the evaluated toolchain and do not attribute it causally to CPU clock frequency differences alone, as architectural factors (superscalar execution, memory bandwidth, cache line widths) contribute simultaneously.
- 2) Unmasking Host Noise-Floor Rank Inversions: On the host x86_64 processor, sub-microsecond execution times for student_b (0.98 μ s), mlp_12f (1.00 μ s), and student_a (1.02 μ s) were compressed within a narrow 40 ns window dominated by OS scheduler jitter and interpreter dispatch overhead. This caused a spurious rank inversion where student_b (328 params) appeared faster than student_a (176 params). Physical microcontroller execution cleanly unmasks this artifact, establishing that student_a is 8.41 μ s faster (11.53% speedup) than student_b on bare metal.

VI. Memory Subsystems and Runtime Determinism

A. RQ4: Memory Partitioning and Arena Sizing

Table V details the physical memory partitioning:

TABLE V
Microcontroller Memory and Flash Partitioning Breakdown

Memory Subsystem Region	Total Capacity	Allocated / Used	% Utilization
Physical SPI Flash Chip	4,194,304 B	330,153 B	7.87%
Application Flash Partition	1,310,720 B	330,153 B	25.19%
Internal On-Chip Static SRAM	327,680 B	61,944 B	18.90%
Statically Allocated Tensor Arena	8,192 B	916 B	11.18%
Free Dynamic Heap (Inference)	237,452 B	0 B (Allocated)	0.00%

- Flash Memory Footprint: The complete firmware binary (incorporating FreeRTOS, Arduino runtime, TFLite Micro interpreter, reference kernels, and all 4 embedded model byte arrays) occupies 330,153 Bytes (25.19% of the 1.25 MB application partition, 7.87% of physical Flash). Storing four sub-4 KB models simultaneously consumes < 15 KB of total Flash storage.
- Static SRAM Allocation: Static global data, peripheral buffers, and runtime stacks require 61,944 Bytes (18.90% of 320 KB internal SRAM).
- Tensor Arena Allocator Commitment: TFLite Micro requires a dedicated tensor arena buffer. We statically allocated an 8,192 Byte arena in internal SRAM. Querying `interpreter->arena_used_bytes()` confirmed that the runtime allocator committed exactly **916** Bytes for input, output, intermediate activation buffers, and runtime context structs across all models. The static arena provides **88.82%** safety headroom (7,276 B uncommitted).
- Dynamic Heap Stability: Querying `esp_get_free_heap_size()` recorded 237,452 Bytes of free heap at initialization, remaining completely invariant throughout all 25,200 inference invocations. The runtime executes with 0 Bytes of dynamic heap allocation and zero memory leakage.

VII. Related Work

Benchmarking machine learning on microcontrollers has gained substantial attention. Banbury et al. [4] established MLPerf Tiny, providing standardized rules for cross-hardware embedded benchmarking across vision, speech, and anomaly detection. David et al. [6] introduced TensorFlow Lite Micro, detailing interpreter architecture and static memory arena management. Lin et al. [8], [9] developed MCUNet, utilizing neural architecture search and memory-efficient patch execution for vision tasks on ARM Cortex-M7 devices. Liberis et al. [12] explored constrained architecture search (μ NAS) on microcontrollers.

In the context of the ESP32 platform, Schizas et al. [5] evaluated dense MLPs for industrial vibration monitoring, though timing loops incorporated serial I/O overhead. Imteaj et al. [7] and Nguyen et al. [10] benchmarked

embedded frameworks across microcontrollers but averaged small sample sizes ($N < 100$) without full percentile distributions. Puranik et al. [11] benchmarked vectorized deep networks on ESP32-S3 using ESP-NN assembly extensions. Blalock et al. [13] surveyed pruning literature, demonstrating that theoretical weight sparsity often fails to accelerate hardware execution without dedicated run-times.

Our work complements these studies by providing the first empirical, publication-grade characterization of sub-4 KB INT8 TinyML models on physical ESP32-D0WD-V3 silicon, combining high-sample-count statistical rigor ($N = 24,000$), zero-I/O in-RAM timing, exact tensor arena memory accounting, and host-to-silicon translation analysis.

VIII. Threats to Validity and Limitations

We explicitly state the following scientific boundaries:

- 1) Single Silicon Target: Experiments were conducted on the dual-core Xtensa LX6 ESP32-D0WD-V3; results do not directly generalize across other microcontroller families (e.g., ARM Cortex-M, RISC-V) or vector-accelerated variants (ESP32-S3).
- 2) Model Topology Focus: Evaluations focused on sub-4 KB Multi-Layer Perceptrons; convolutional and recurrent architectures may exhibit different memory and compute dynamics.
- 3) Software Stack Dependency: Benchmarks utilize the portable reference C++ integer kernel in TFLM; assembly-optimized libraries (e.g., ESP-NN) may achieve lower latencies.
- 4) Absence of Physical Energy Profiling: We report execution latency and memory footprints; physical energy and power dissipation were not instrumented via hardware current shunts.
- 5) Empirical Latency vs. Formal WCET: The maximum observed latency (102 μ s) reflects empirical benchmark distributions and does not establish a formal static WCET bound.
- 6) Isolated Kernel Scope: Timings capture pure model inference and exclude physical sensor ADC acquisition and CAN-bus communication delays.

IX. Reproducibility and Artifact Availability

All firmware source code, PlatformIO configuration scripts, compiled TFLite FlatBuffers, C-byte array headers, and raw serial logs are open-sourced in the project repository. The firmware can be compiled and deployed using PlatformIO by executing `piorun-tupload targeting env: esp32_devkit`. Raw benchmark execution logs and parsed CSV datasets are archived under `phase5/measurements/`.

X. Conclusion

This paper presented an empirical on-device characterization and latency profiling of four serialized FULL_INT8 TinyML models deployed on physical ESP32-D0WD-V3 silicon. Across 24,000 measured single-sample inferences,

on-device execution ranged from 64.55 μ s to 89.90 μ s, demonstrating an observed monotonic relationship with parameter count ($R^2 = 0.963$) and proving that structural knowledge distillation delivers a genuine 28.20% execution speedup on embedded ALUs. We quantified an observed host-to-microcontroller latency slowdown between $62.87\times$ and $76.77\times$, unmasking host-side sub-microsecond noise-floor rank inversions. Furthermore, memory profiling verified that TensorFlow Lite Micro commits exactly 916 Bytes of tensor arena memory with zero dynamic heap allocations across 25,200 executions. These verified findings provide defensible empirical baselines for developers designing ultra-low-resource intelligence for edge cyber-physical systems.

References

- [1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2019.
- [2] P. P. Ray, "A review on tinyml: State-of-the-art, research challenges, and future directions," *Journal of Systems Architecture*, vol. 126, p. 102468, 2022.
- [3] B. Saha, S. Dey, and A. Roy, "Machine learning on edge devices: A survey on tinyml," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–38, 2022.
- [4] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Natarajan, S. Lam, M. Lam et al., "Benchmarking tinyml systems: Challenges and direction," *IEEE Micro*, vol. 41, no. 6, pp. 30–39, 2021.
- [5] N. Schizas, A. Karras, C. Karras, and S. Sioutas, "Tinyml for industrial condition monitoring: An esp32 on-device implementation," *IEEE Access*, vol. 10, pp. 81 442–81 454, 2022.
- [6] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, R. Marculescu et al., "Tensorflow lite micro: Embedded machine learning on tinyml systems," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
- [7] A. Imteaj, M. Amini, and U. Thakker, "Benchmarking edge ai execution frameworks on low-power microcontrollers," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 420–10 432, 2023.
- [8] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11 711–11 722.
- [9] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "Mcunetv2: Memory-efficient patch-based inference for tiny deep learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 879–891.
- [10] V.-D. Nguyen, M.-T. Le, and D.-S. Kim, "Empirical latency and memory characterization of quantized neural networks on embedded iot nodes," *Sensors*, vol. 24, no. 5, p. 1520, 2024.
- [11] A. Puranik, R. Deshmukh, and S. Sapatnekar, "Benchmarking quantized deep neural networks on vectorized microcontrollers," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 3, pp. 1241–1245, 2024.
- [12] E. Liberis, L. Dudziak, and N. D. Lane, " μ nas: Constrained neural architecture search for microcontrollers," in *Proceedings of the 1st Workshop on Machine Learning and Systems*, 2021, pp. 1–7.
- [13] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Gutttag, "What is the state of neural network pruning?" in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 129–146.